DAY 05 OF 30

# Ingest & Transform Data
# Loading Patterns & Pipelines

Master full vs incremental loads, Copy Activity, OneLake Shortcuts, Mirroring, and bad data handling for the DP-700 exam

- Full vs Incremental Load
- Copy Activity Deep Dive
- Shortcuts vs Mirroring
- Choose the Right Data Store
- Duplicate & Null Handling
- 3 Practice Questions

INGEST & TRANSFORM → Design and implement full and incremental data loads

# Full Load vs Incremental Load

## 📦 Full Load

TRUNCATE & RELOAD ALL DATA EVERY RUN

- Simple — no watermark tracking needed
- Always 100% consistent with source
- Expensive for large tables
- Long run time, high compute cost

Best for: Small reference/dimension tables, initial loads

## Incremental Load

LOAD ONLY NEW/CHANGED RECORDS VIA WATERMARK

- Uses **watermark column** (last_modified_date, id)
- Only processes changed rows — fast & efficient
- Scales to hundreds of millions of rows
- Doesn't catch deletes by default ⚠️

Best for: Large transactional tables, daily pipelines

💡 **Exam signal word: "truncate"**

If the question says truncate-and-reload or mentions small lookup tables → Full Load is the answer.

## 🔄 CDC / Mirroring

CAPTURES INSERTS + UPDATES + DELETES

- Database log-level change capture
- Fabric Mirroring: near-real-time replication
- No pipeline required for CDC

Best for: Real-time replication from Azure SQL / Cosmos / Snowflake

LOADING PATTERNS → Prepare data for loading into a dimensional model

# Loading Into a Star Schema

6 steps the DP-700 expects you to know in order

### 1 Denormalize

Flatten normalized source tables using JOINs. PySpark JOIN or T-SQL JOIN across Bronze tables into Silver.

### 2 Deduplicate

`dropDuplicates()` for exact. `ROW_NUMBER() OVER (PARTITION BY key ORDER BY date DESC)` for soft dupes.

### 3 Surrogate Keys

Generate integer PKs for dimension tables. Use `monotonically_increasing_id()` in PySpark or `IDENTITY` in T-SQL.

### 4 Handle Nulls

Fill with defaults: `fillna({"revenue": 0})` or drop rows where key columns are null: `dropna(subset=["id"])`

### 5 SCD Type 2

Track historical dimension changes. Add **valid_from**, **valid_to**, **is_current** columns. Use Delta MERGE to expire old rows and insert new ones.

### 6 Load Fact Table

Look up surrogate keys from dimension tables, aggregate measures, write to Gold fact table. Append mode for incremental.

### SCD Type 1 vs Type 2

Type 1 = overwrite (lose history)
Type 2 = add new row + expire old (keep history)
DP-700 favourite question: "preserve history" → always SCD Type 2

```sql
-- SCD Type 2 check pattern
SELECT customer_id, email,
  -- Expire old row
  CASE WHEN src.email != tgt.email
    THEN 'false' ELSE tgt.is_current
  END AS is_current,
  CURRENT_DATE() AS valid_to
FROM gold.dim_customer tgt
JOIN source src ON
  tgt.customer_id = src.customer_id
  AND tgt.is_current = true
```

PIPELINES → Ingest data by using pipelines — Copy Activity configuration

# Copy Activity: Source → Sink

The primary ingestion mechanism in Fabric Data Pipelines

### Source

- Connection & linked service
- Table / query / file path
- File format (CSV, Parquet, JSON)
- Partition options (parallelism)

### Sink

- Destination store
- Write behaviour
- Pre-copy script (truncate)
- File naming pattern

### Mapping Tab

Column-to-column schema mapping and data type conversions. Auto-detect or manual. Handles source/sink column name mismatches.

## Sink Write Behaviours

| Mode | Behaviour | Use When |
|---|---|---|
| **Append** | Add rows, keep existing | Incremental → Lakehouse |
| **Overwrite** | Truncate then write | Full load → Lakehouse table |
| **Upsert** | Insert new, update existing | SCD Type 1, merge pattern |
| **Insert** | Insert only, fail on duplicate key | Warehouse fact table |

**Exam Trap: Pre-copy Script**

The "Pre-copy script" in the Sink runs BEFORE the copy. Use it to TRUNCATE the target table for full loads — not a separate pipeline activity.

INGEST & TRANSFORM → Choose an appropriate data store

# Lakehouse vs Warehouse vs Eventhouse

### 🏠 Lakehouse

- Delta Lake on OneLake
- PySpark + SQL endpoint
- Schema-on-read
- Best for: Bronze/Silver/Gold ELT
- DirectLake for Power BI

### 🏛️ Warehouse

- SQL engine, T-SQL only
- ACID transactions
- Schema-on-write
- Best for: Star schema, reporting
- Views, procedures, functions

### ⚡ Eventhouse

- KQL Database inside
- Sub-second on billions of rows
- Time-series & telemetry
- Best for: IoT, logs, streaming
- Kusto Query Language (KQL)

### 🔄 Mirrored DB

- Replicated via CDC
- Azure SQL, Cosmos, Snowflake
- Near-real-time in OneLake
- Best for: Operational analytics
- No pipeline ingestion needed

⚠️ **Exam Trap: Lakehouse SQL Endpoint is READ-ONLY T-SQL.**

You cannot INSERT/UPDATE/DELETE via the SQL Analytics Endpoint of a Lakehouse. For full T-SQL write support, use a Warehouse. For PySpark writes, use the Lakehouse Spark interface.

INGEST DATA → Create and manage OneLake shortcuts • Implement mirroring

# OneLake Shortcuts vs Mirroring

Two ways to access external data in Fabric — without a full pipeline copy

## 🔗 OneLake Shortcuts

VIRTUAL POINTER — DATA STAYS EXTERNAL

- No physical data copy — pointer only
- Data lives in ADLS Gen2, S3, GCS, or other Lakehouse
- Read-only access in Fabric
- Auth: User identity OR service principal (delegated)
- Supports Delta, Parquet, CSV formats

Supported: ADLS Gen2 • Amazon S3 • GCS • OneLake

## 🔄 Fabric Mirroring

CONTINUOUS CDC REPLICA INTO ONELAKE

- Data IS replicated into OneLake in Delta format
- Near real-time via Change Data Capture (CDC)
- No pipeline required — automatic sync
- Enables full Spark, SQL endpoint, Power BI analytics
- Read-only in Fabric (source remains writable)

Supported: Azure SQL DB • Cosmos DB • Snowflake • Azure SQL MI

| | Shortcut | Mirroring |
|---|---|---|
| Data location | External (pointer only) | Replicated into OneLake |
| Latency | Depends on external source | Near real-time (CDC) |
| Azure SQL / Cosmos DB | ❌ Not supported | ✅ Yes |
| ADLS Gen2 / S3 / GCS | ✅ Yes | ❌ Not supported |

TRANSFORM DATA → Choose between PySpark, SQL, KQL and Dataflow Gen2

# Which Transformation Tool When?

| Tool | Language | Best For | Runs On | Schedulable |
|---|---|---|---|---|
| **PySpark (Notebook)** | Python / Spark SQL | Large-scale ELT, complex logic, ML feature engineering | Spark cluster | Via Pipeline only |
| **T-SQL (Warehouse)** | T-SQL | Relational transforms, star schema loads, stored procedures | SQL engine | Via Pipeline only |
| **KQL (Eventhouse)** | Kusto Query Language | Time-series, telemetry, streaming analytics | KQL engine | Via Pipeline only |
| **Dataflow Gen2** | Power Query M | No-code transforms, citizen data prep, small-medium datasets | M engine | ✅ **Self-schedule** |

💡 **Key rule: Only Dataflow Gen2 can self-schedule**

PySpark notebooks, T-SQL scripts, and KQL queries all require a Pipeline wrapper to schedule them. This is a top DP-700 exam question.

📊 **Group & Aggregate in each tool**

PySpark: `.groupBy().agg()` | T-SQL: `GROUP BY` + `SUM/COUNT` | KQL: `summarize` | M: Group By step

TRANSFORM DATA → Handle duplicate, missing, and late-arriving data

# Duplicate, Missing & Late-Arriving Data

### 🗂️ Duplicates

**Exact:** `df.dropDuplicates()`

**Soft (keep latest):**
ROW_NUMBER() OVER
(PARTITION BY key
ORDER BY modified_date DESC)
Filter WHERE rn = 1

### 🔲 Missing / Nulls

**Fill:** `fillna({"rev":0,"region":"Unknown"})`

**Drop nulls on key cols:**
`dropna(subset=["customer_id"])`

**T-SQL:**
`COALESCE(revenue, 0)`
`ISNULL(region, 'Unknown')`

### ⏰ Late-Arriving Data

Data arrives AFTER its event time window was processed.

**Spark Streaming Watermark**

```
.withWatermark("event_time", "2 hours")
```

Accept late records up to 2h late

**Reprocessing**

Re-run the affected pipeline window when late data arrives

**SCD Type 2**

Late dimension changes tracked with valid_from / valid_to

QUICK REFERENCE → Day 05 cheat sheet — save this slide

# Ingestion Decision Cheat Sheet

| Exam Scenario | Answer | Why |
|---|---|---|
| Load 5M rows daily, source has `last_modified` column | Incremental Load + Watermark | Only process changed rows — efficient & scalable |
| Small 10K-row country lookup table, needs daily refresh | Full Load | Simple, always consistent, small table = low cost |
| Bring Azure SQL DB data into Fabric with near-real-time freshness | Fabric Mirroring | CDC replication — no pipeline, near-real-time |
| Access ADLS Gen2 data from Fabric without copying it | OneLake Shortcut | Virtual pointer — data stays in ADLS |
| Customer email changed — must preserve historical value | SCD Type 2 | Add new row, expire old with valid_to + is_current=false |
| Remove exact duplicate rows before loading to Silver | dropDuplicates() | PySpark native dedup on all columns |
| Streaming events arrive up to 90 mins late — how to handle? | Watermark tolerance | .withWatermark("event_time", "90 minutes") |
| Full T-SQL write support needed (INSERT/UPDATE/DELETE) | Data Warehouse | Lakehouse SQL endpoint is READ-ONLY |

# Ingestion mastered.
# 30–35% of DP-700 covered.

Full load · Incremental · CDC · Copy Activity · Shortcuts · Mirroring · Dedup · Late data — you now know all of it.

🔔 **Day 06 Tomorrow: Ingest Streaming Data — Eventstreams, KQL & Spark Structured Streaming**

**Sayan Banerjee** • AI, BI & Analytics Consultant
Microsoft Certified: Fabric Data Engineer Associate (DP-700)